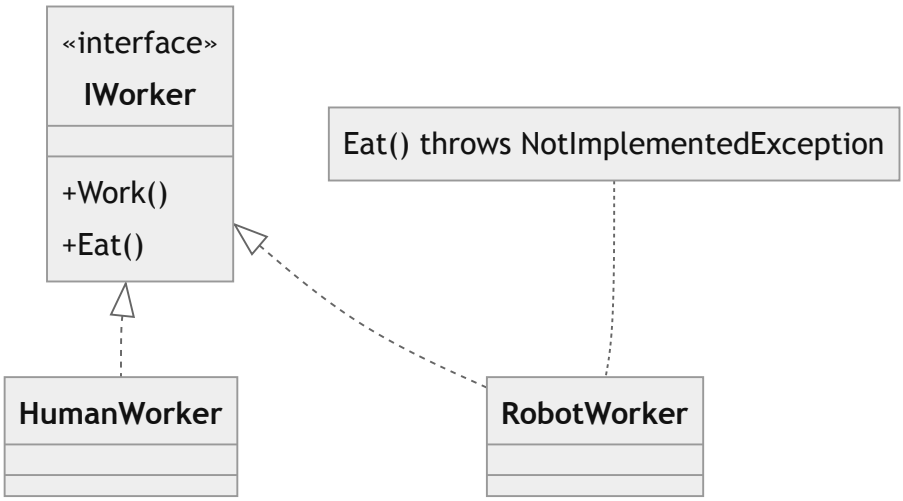
# MCQ Practice

Choose the best answer for each question. Name: _____

## Q1

In the design below, `RobotWorker` cannot eat, so its `Eat()` throws `NotImplementedException`. Which SOLID principle does the design of the `IWorker` interface violate?




**a.** Single Responsibility Principle.
**b.** Open-Closed Principle.
**c.** Dependency Inversion Principle.
**d.** Liskov Substitution Principle.
**e.** Interface Segregation Principle.

## Q2

What does `new Car();` print?

```
class Vehicle {
    public Vehicle(int wheels) {
        Console.Write(wheels);
    }
}

class Car : Vehicle {
    public Car() : base(4) {
        Console.Write("Car");
    }
}
```

**a.** Car4
**b.** Car
**c.** 4
**d.** Compile-time error.
**e.** 4Car

## Q3

Methods in a class are grouped because they must be called in a certain order (e.g. `CheckPassword()` then `LoadMenu()`), even though they work on different data. This is an example of:

**a.** Procedural cohesion
**b.** Sequential cohesion
**c.** Functional cohesion
**d.** Logical cohesion
**e.** Communicational cohesion

---

## Q4

What does the code below print?

```
void AddTen(int x) {
    x += 10;
}

void AddTenRef(ref int x) {
    x += 10;
}

int a = 5;
int b = 5;
AddTen(a);
AddTenRef(ref b);
Console.Write(a + " " + b);
```

**a.** `5 5`
**b.** `15 15`
**c.** `15 5`
**d.** `5 15`
**e.** Compile-time error.

---

## Q5

Which of the following is NOT one of the SOLID principles?

**a.** Open-Closed Principle.
**b.** Liskov Substitution Principle.
**c.** Don't Repeat Yourself.
**d.** Interface Segregation Principle.
**e.** Dependency Inversion Principle.

---

## Q6

In a booking app, `BookingService` depends on `IFlightRepository`. Switching from `InMemoryFlightRepository` to `SqlFlightRepository` only requires changing one line in `Main`. Which SOLID principle mainly makes this possible?

**a.** Single Responsibility Principle.
**b.** Liskov Substitution Principle.
**c.** Interface Segregation Principle.
**d.** Dependency Inversion Principle.
**e.** None of the above.

---

## Q7

A well-designed system should aim for:

**a.** Low cohesion and low coupling.
**b.** Low cohesion and high coupling.
**c.** High cohesion and low coupling.
**d.** High cohesion and high coupling.
**e.** Cohesion and coupling do not matter.

---

## Q8

What does the code below do?

```
class Traveller {
    public virtual double GetDiscount() {
        return 0;
    }
}

class GoldTraveller : Traveller {
    public override double GetDiscount() {
        return 0.2;
    }
}

List<Traveller> travellers = new List<Traveller>();
travellers.Add(new Traveller());
travellers.Add(new GoldTraveller());

foreach (Traveller t in travellers) {
    Console.Write(t.GetDiscount() + " ");
}
```

**a.** Compile-time error, because a `List<Traveller>` cannot store a `GoldTraveller`.
**b.** Prints `0 0`, because the list only knows about `Traveller`.
**c.** Prints `0 0.2`.
**d.** Throws `InvalidCastException` at run time.
**e.** Prints `0.2 0.2`.

---

## Q9

Cohesion measures:

**a.** How many classes depend on a class.
**b.** How fast a class runs.
**c.** How many lines of code a class has.
**d.** How many interfaces a class implements.
**e.** How closely related the responsibilities inside a single class or module are.

---

## Q10

The interface `ISuperMachine` declares `Print()`, `Scan()` and `Fax()`. A team adds a new method `Staple()` to it. What is the main problem in a large codebase?

**a.** The interface becomes slower.
**b.** Nothing, interfaces can grow freely.
**c.** Only `SuperMachine` must change.
**d.** Every class implementing `ISuperMachine`, even simple printers, must now change.
**e.** The interface can no longer be used as a parameter type.

---

## Q11

In C#, which statement about `ref` and `out` parameters is correct?

**a.** `ref` and `out` behave exactly the same.
**b.** A `ref` variable must be assigned before it is passed in; an `out` parameter must be assigned inside the method before it returns.
**c.** An `out` variable must be assigned before it is passed in; a `ref` parameter must be assigned inside the method.
**d.** Both only work with reference types such as classes.
**e.** Both pass a copy of the value, so the caller never sees changes.

---

## Q12

`Print(report, true)` prints a PDF and `Print(report, false)` prints plain text. The caller passes a flag that decides what the method does. Which type of coupling is this?

**a.** Data coupling
**b.** Stamp coupling
**c.** Control coupling
**d.** Content coupling
**e.** Common coupling

---

## Q13

`Penguin` inherits from `Bird`, and `Penguin.Fly()` throws an exception. Which redesign best fixes the problem?

**a.** Wrap every call to `Fly()` in a try-catch.
**b.** Move `Fly()` into a separate `FlyingBird` class (or `IFlyable` interface) that only flying birds use.
**c.** Make `Penguin.Fly()` do nothing and return silently.
**d.** Make `Bird` a sealed class.
**e.** Check `if (bird is Penguin)` before every call to `Fly()`.

---

## Q14

What does the code below print?

```
class A {
    public void Show() {
        Console.Write("A");
    }
}

class B : A {
    public new void Show() {
        Console.Write("B");
    }
}

A obj = new B();
obj.Show();
```

**a.** A
**b.** B
**c.** AB
**d.** BA
**e.** Compile-time error.

---

## Q15

`Notification` depends on the `IMessage` interface instead of the `Email` class. What effect does this have?

**a.** It reduces coupling, so Notification works with any IMessage (Email, WhatsApp) without changes.
**b.** It increases coupling between Notification and Email.
**c.** It increases the cohesion of Email.
**d.** It makes Notification an anemic model.
**e.** It has no effect.

---

## Q16

`BankAccount.Withdraw(amount)` accepts any positive amount. A child class `StudentAccount` overrides it and throws an exception when `amount > 100`. Code that works with `BankAccount` now breaks when given a `StudentAccount`. Which SOLID principle is violated?

**a.** Interface Segregation Principle.
**b.** Single Responsibility Principle.
**c.** Dependency Inversion Principle.
**d.** Liskov Substitution Principle.
**e.** Open-Closed Principle.

---

## Q17

In the code below, `Print()` crashes with a `NullReferenceException` if `Load()` has not been called first. Which type of coupling is this?

```
class SalesReport {
    private List<Sale> sales;

    public void Load(List<Sale> sales) {
        this.sales = sales;
    }
    public void Print() {
        foreach (Sale sale in sales) { /* ... */ }
    }
}

SalesReport report = new SalesReport();
report.Print();
```

**a.** Content coupling
**b.** Common coupling
**c.** Control coupling
**d.** Stamp coupling
**e.** Temporal coupling

---

## Q18

What happens when the code below is compiled?

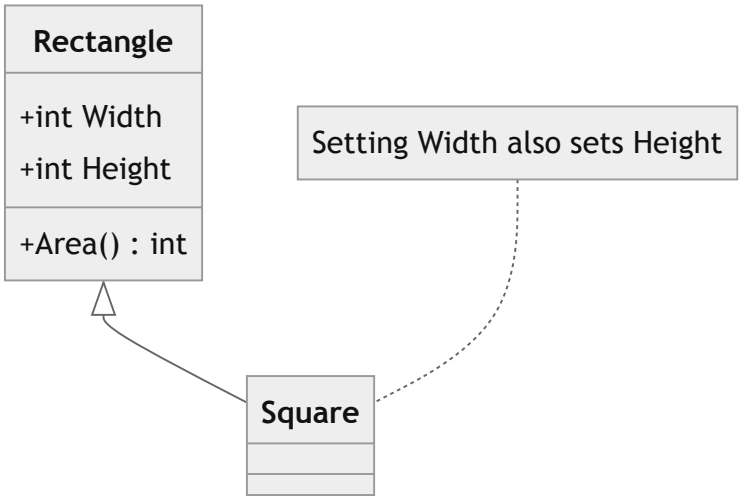
```
void GetAge(out int age) {
    Console.Write("Hello");
}
```

**a.** It compiles, and `age` is 0 after the call.
**b.** It compiles, and `age` keeps the value it had before the call.
**c.** It compiles, but throws an exception at run time.
**d.** It compiles only if the caller initialises `age` first.
**e.** Compile-time error, because an `out` parameter must be assigned before the method returns.

---

## Q19

In the design below, setting `Width` on a `Square` also changes its `Height`. A method that takes a `Rectangle`, sets `Width = 5` and `Height = 4`, then expects `Area() == 20`, fails when given a `Square`. Which SOLID principle is violated?




**a.** Single Responsibility Principle.
**b.** Open-Closed Principle.
**c.** Liskov Substitution Principle.
**d.** Interface Segregation Principle.
**e.** Dependency Inversion Principle.

---

## Q20

A `TaxCalculator` class where every field and method contributes to one task, calculating income tax, is an example of:

**a.** Functional cohesion
**b.** Sequential cohesion
**c.** Communicational cohesion
**d.** Logical cohesion
**e.** Temporal cohesion

---

## Q21

`SendWelcomeEmail(Traveller traveller)` only uses `traveller.Email`, but receives the whole `Traveller` object. Which type of coupling is this?

**a.** Stamp coupling
**b.** Data coupling
**c.** Control coupling
**d.** Common coupling
**e.** Content coupling

---

### Q22

A `Utils` class contains `FormatDate()`, `CalculateTax()` and `SendEmail()`, which have nothing in common. This is an example of:

**a.** Coincidental cohesion
**b.** Logical cohesion
**c.** Temporal cohesion
**d.** Functional cohesion
**e.** Sequential cohesion

---

### Q23

`GoldTraveller` inherits from `Traveller`. What happens with the code below?

```
List<GoldTraveller> golds = new List<GoldTraveller>();
List<Traveller> travellers = golds;
```

**a.** It compiles, because a `GoldTraveller` is a `Traveller`.
**b.** It compiles, but throws `InvalidCastException` at run time.
**c.** It compiles, and `travellers` becomes a copy of `golds`.
**d.** Compile-time error: a `List<GoldTraveller>` is not a `List<Traveller>`, even though a `GoldTraveller` is a `Traveller`.
**e.** None of the above.

---

### Q24

How should the class below be changed to follow the Dependency Inversion Principle?

```
class OrderService {
    private SqlDatabase db = new SqlDatabase();
}
```

**a.** Make `SqlDatabase` a static class.
**b.** Make `OrderService` inherit from `SqlDatabase`.
**c.** Depend on an `IDatabase` interface and receive it through the constructor.
**d.** Copy the database code into `OrderService`.
**e.** Make the `db` field public.

---

### Q25

A `CustomerReport` class has methods that print a customer's details, update the customer's address and calculate the customer's balance. All the methods work on the same customer record. This is an example of:

**a.** Logical cohesion
**b.** Temporal cohesion
**c.** Communicational cohesion
**d.** Coincidental cohesion
**e.** Procedural cohesion

---

## Q26

In C#, when is the method to call decided for overloading and for overriding?

**a.** Overloading is decided at compile time; overriding is decided at run time.
**b.** Both are decided at compile time.
**c.** Both are decided at run time.
**d.** Overloading is decided at run time; overriding is decided at compile time.
**e.** Neither is decided until the method is called twice.

---

## Q27

`TravellerController.Book()` checks the traveller's points, subtracts the cost, and prints "Booked!" with `Console.WriteLine`, while `Traveller` only has `{ get; set; }` properties. Which statement is the most accurate?

**a.** This is the recommended "fat model, thin controller" design.
**b.** The controller is a good example of the Interface Segregation Principle.
**c.** This is a fat controller with an anemic model: the controller does the Model's job and the View's job.
**d.** This follows MVC because all the code is in one place.
**e.** None of the above.

---

## Q28

Which list orders cohesion from WORST to BEST?

**a.** Functional, Sequential, Communicational, Procedural, Temporal, Logical, Coincidental.
**b.** Coincidental, Temporal, Logical, Functional, Procedural, Communicational, Sequential.
**c.** Logical, Coincidental, Temporal, Procedural, Sequential, Communicational, Functional.
**d.** Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional.
**e.** Temporal, Coincidental, Logical, Procedural, Functional, Sequential, Communicational.

---

## Q29

A `DiscountCalculator` has a `switch` on the customer type that must be edited every time a new customer type is added. What is the best way to fix it?

**a.** Add a comment reminding developers to update the switch.
**b.** Replace the switch with a chain of if-else statements.
**c.** Move the switch into the Main method.
**d.** Make the customer type a public field so any class can change it.
**e.** Give each customer type its own class that overrides a virtual `GetDiscount()` method.

---

### Q30

`InputHandler.Handle(int source)` reads from the keyboard, a file or the network depending on the `source` flag. The parts are grouped because they are the same kind of thing (input), not because they work together. This is an example of:

**a.** Coincidental cohesion
**b.** Logical cohesion
**c.** Temporal cohesion
**d.** Sequential cohesion
**e.** Functional cohesion

---

### Q31

Class `A` changes a private field of class `B` directly using reflection. Which type of coupling is this?

**a.** Data coupling
**b.** Stamp coupling
**c.** Control coupling
**d.** Content coupling
**e.** Common coupling

---

### Q32

A `Startup` class contains `InitLogger()`, `LoadConfig()` and `OpenDatabase()`. They are grouped only because they all run when the program starts. This is an example of:

**a.** Coincidental cohesion
**b.** Logical cohesion
**c.** Temporal cohesion
**d.** Communicational cohesion
**e.** Functional cohesion

---

### Q33

Which pair of methods is a valid overload in the same class?

**a.** `int Add(int a, int b)` and `double Add(int a, int b)`
**b.** `void Print(int x)` and `void Print(int y)`
**c.** `void Print(int x)` and `void Print(string x)`
**d.** `void Print(int x)` and `void PrintInt(int x)`
**e.** None of the above.

---

### Q34

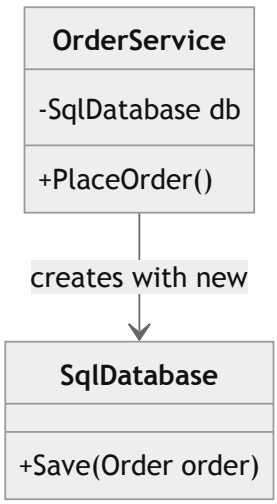
A `DiscountCalculator` has a `switch` on the customer type ("Regular", "Silver", "Gold"). Every time a new customer type is added, a developer must edit this `switch`. Which SOLID principle is violated?

**a.** Single Responsibility Principle.
**b.** Open-Closed Principle.
**c.** Liskov Substitution Principle.
**d.** Interface Segregation Principle.
**e.** Dependency Inversion Principle.

---

## Q35

In the design below, `OrderService` creates its own `SqlDatabase` with `new SqlDatabase()`. Which SOLID principle does this violate most directly?

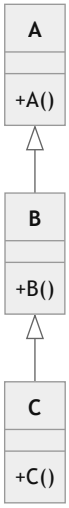



**a.** Liskov Substitution Principle.
**b.** Interface Segregation Principle.
**c.** Open-Closed Principle.
**d.** Single Responsibility Principle.
**e.** Dependency Inversion Principle.

---

## Q36

Each constructor below writes its own class name with `Console.Write`. What does `new C();` print?




**a.** C
**b.** CBA
**c.** ABC
**d.** AC
**e.** Compile-time error.

---

## Q37

Which statement best describes "one reason to change" in the Single Responsibility Principle?

**a.** A class should have only one method.
**b.** A class should have only one field.
**c.** A class should have only one job, so only changes to that job affect it.
**d.** A class should be changed at most once.
**e.** A class should never be changed after release.

---

## Q38

`GoldTraveller` inherits from `Traveller`, and `golds` is a `List<GoldTraveller>`. Which line compiles?

**a.** `List<Traveller> a = golds;`
**b.** `IEnumerable<Traveller> b = golds;`
**c.** `List<object> c = golds;`
**d.** `GoldTraveller d = golds;`
**e.** None of the above.

---

## Q39

What does the code below print?

```
class Animal { }
class Dog : Animal { }

void Speak(Animal a) {
    Console.Write("Animal ");
}

void Speak(Dog d) {
    Console.Write("Dog ");
}

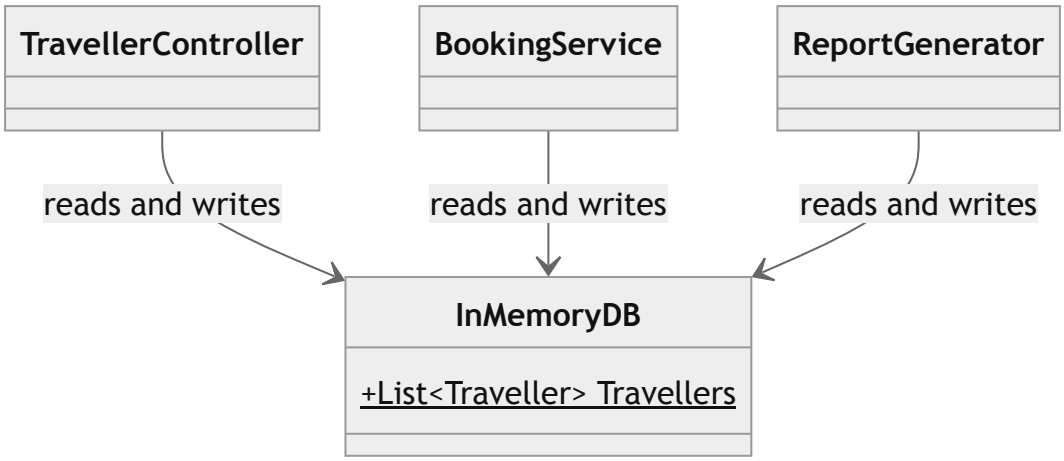
Animal pet = new Dog();
Speak(pet);
Speak(new Dog());
```

**a.** `Dog Dog`
**b.** `Animal Dog`
**c.** `Animal Animal`
**d.** `Dog Animal`
**e.** Compile-time error, because the call is ambiguous.

---

## Q40

In the design below, several classes read and write `InMemoryDB.Travellers`, a `public static` list. Which type of coupling is this?




**a.** Data coupling
**b.** Common coupling
**c.** Stamp coupling
**d.** Control coupling
**e.** Content coupling

---

## Q41

Which is a typical consequence of tight coupling?

**a.** Classes can easily be tested in isolation.
**b.** Classes can easily be reused in other projects.
**c.** Changes always stay inside one class.
**d.** A change in one class forces changes in other classes, and testing in isolation becomes hard.
**e.** The program always runs faster.

---

## Q42

What happens when the code below is compiled?

```
class Person {
    public Person(string name) { }
}

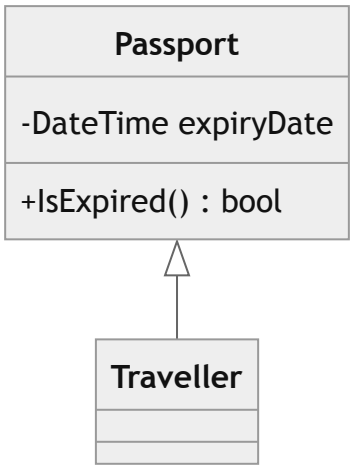
class Student : Person {
    public Student() { }
}
```

**a.** It compiles, and Person's constructor is skipped.
**b.** It compiles, and name is set to null.
**c.** It compiles, but throws an exception at run time.
**d.** Compile-time error, because Person has no parameterless constructor and Student does not call base(...).
**e.** None of the above.

---

### Q43

A developer designs the classes below so that `Traveller` can reuse `IsExpired()`. What is the better design?




**a.** Make `Passport` inherit from `Traveller`.
**b.** Make `IsExpired()` static.
**c.** Copy `IsExpired()` into `Traveller`.
**d.** Make `Passport` an interface with all of Traveller's methods.
**e.** Use composition: `Traveller` has a private `Passport` field and calls `passport.IsExpired()`.

---

### Q44

A class has `ReadFile()`, `ParseLines()` and `SaveRecords()`, where the output of each method is the input of the next, like an assembly line. This is an example of:

**a.** Procedural cohesion
**b.** Communicational cohesion
**c.** Temporal cohesion
**d.** Sequential cohesion
**e.** Coincidental cohesion

---

### Q45

To override a method in C#, which keywords are needed in the parent and in the child?

**a.** Parent: nothing. Child: `override`.
**b.** Parent: `override`. Child: `virtual`.
**c.** Parent: `new`. Child: `override`.
**d.** Parent: `virtual` (or `abstract`). Child: `override`.
**e.** Parent: `static`. Child: `override`.

---

## Q46

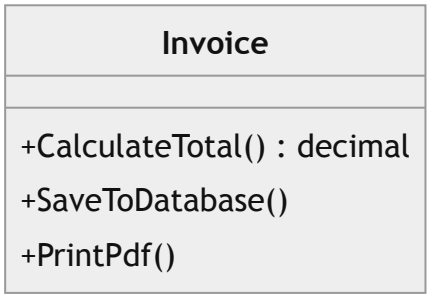
A `ReportGenerator` class loads sales data from the database, formats it as HTML, and emails it to the manager. Which description fits best?

**a.** Low cohesion and a violation of SRP; it should be split into separate classes for loading, formatting and sending.
**b.** High cohesion, and it follows SRP.
**c.** Loose coupling, and it follows ISP.
**d.** Functional cohesion, and it follows OCP.
**e.** None of the above.

---

## Q47

Which SOLID principle does the `Invoice` class below violate?




**a.** Single Responsibility Principle.
**b.** Open-Closed Principle.
**c.** Liskov Substitution Principle.
**d.** Interface Segregation Principle.
**e.** Dependency Inversion Principle.

---

## Q48

What does `new B();` print?

```
class A {
    public A() {
        Show();
    }
    public virtual void Show() {
        Console.Write("A");
    }
}

class B : A {
    private string name = "B";

    public B() {
        name = "C";
    }
    public override void Show() {
        Console.Write(name);
    }
}
```

**a.** A
**b.** B
**c.** C
**d.** Nothing, because name is still null.
**e.** Compile-time error.

---